SQL DATABASE PROJECT

Bookstore Management System

Data Analysis & Querying using Microsoft SQL Server

T-SQL | Joins | CTEs | Window Functions | Aggregation

Project Overview

Database

Microsoft SQL Server database named 'project' with 3 core business tables: Books, Customers & Orders

Queries Written

11 basic queries + 8 advanced queries + 2 special SQL problems covering hierarchy & window functions

Skills Covered

SELECT, JOINs, GROUP BY, HAVING, SUM/AVG/COUNT, CTEs, Recursive CTEs, Window Functions

Database Schema

Books	Book_ID · Title · Author · Genre · Price · Stock · Published_Year
Customers	Customer_ID · Name · City · Country
Orders	Order_ID · Book_ID · Customer_ID · Quantity · Total_Amount · Order_Date
Employee_Table	EmployeeID · EmployeeName · DepartmentID · ManagerID
Sample_1	X · Y · Z

Basic Queries — Part 1

Q1

Fiction Genre Books

```
SELECT * FROM Books
WHERE Genre = 'Fiction'
```

Q2

Books After 1950

```
SELECT * FROM Books
WHERE Published_Year > 1950
```

Q3

Customers from Canada

```
SELECT * FROM Customers
WHERE Country = 'Canada'
```

Q4

Orders in Nov 2023

```
SELECT * FROM Orders WHERE
Order_Date BETWEEN '2023-11-01'
AND '2023-11-30'
```

Q5

Total Stock Available

```
SELECT SUM(Stock) AS Total_Stock
FROM Books
```

Q6

Most Expensive Book

```
SELECT TOP 1 * FROM Books
ORDER BY Price DESC
```

Basic Queries — Part 2

Q7

Orders with Quantity > 1

```
SELECT * FROM Orders
WHERE Quantity > 1
```

Q8

Orders Exceeding $20

```
SELECT * FROM Orders
WHERE Total_Amount > 20
```

Q9

All Unique Genres

```
SELECT DISTINCT Genre FROM
Books
```

Q10

Book with Lowest Stock

```
SELECT TOP 1 * FROM Books
ORDER BY Stock ASC
```

Q11

Total Revenue

```
SELECT SUM(Total_Amount)
AS Revenue FROM Orders
```

Advanced Queries — Part 1

AQ1

JOIN + GROUP BY

Books Sold per Genre

```
SELECT b.Genre, SUM(o.Quantity) AS Total_Books_Sold
FROM Orders o
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY b.Genre
```

AQ2

AVG()

Avg Price — Fantasy Genre

```
SELECT AVG(Price) AS Average_Price
FROM Books
WHERE Genre = 'Fantasy'
```

AQ3

HAVING

Customers with 2+ Orders

```
SELECT o.Customer_ID, c.Name,
COUNT(o.Order_ID) AS order_count
FROM Orders o
JOIN Customers c ON o.Customer_ID=c.Customer_ID
GROUP BY o.Customer_ID, c.Name
HAVING COUNT(o.Order_ID) >= 2
```

AQ4

TOP 1 + JOIN

Most Ordered Book

```
SELECT TOP 1 o.Book_ID, b.Title,
COUNT(o.Order_ID) AS order_count
FROM Orders o
JOIN Books b ON o.Book_ID = b.Book_ID
GROUP BY o.Book_ID, b.Title
ORDER BY order_count DESC
```

Advanced Queries — Part 2

AQ5

TOP 3

Top 3 Fantasy Books by Price

```
SELECT TOP 3 * FROM Books
WHERE Genre = 'Fantasy'
ORDER BY Price DESC
```

AQ6

GROUP BY Author

Books Sold by Author

```
SELECT b.Author,
SUM(o.Quantity) AS total_books_sold
FROM Orders o
JOIN Books b ON o.Book_ID=b.Book_ID
GROUP BY b.Author
```

AQ7

DISTINCT + JOIN

Cities — Spent Over $30

```
SELECT DISTINCT c.City, Total_Amount
FROM Orders o
JOIN Customers c
ON o.Customer_ID = c.Customer_ID
WHERE o.Total_Amount > 30
```

AQ8

TOP 1 + SUM

Top Spending Customer

```
SELECT TOP 1 c.Customer_ID, c.Name,
SUM(o.Total_Amount) AS Total_Spent
FROM Orders o
JOIN Customers c
ON o.Customer_ID = c.Customer_ID
GROUP BY c.Customer_ID, c.Name
ORDER BY Total_Spent DESC
```

Employee Hierarchy — Recursive CTE

recursive_cte.sql

```sql
WITH cte_a AS (
  SELECT EmployeeID, EmployeeName,
    ManagerID, 0 AS EmployeeLevel
  FROM Employee_Table
  WHERE ManagerID IS NULL   -- Top manager

  UNION ALL

  SELECT emp.EmployeeID, emp.EmployeeName,
    emp.ManagerID, EmployeeLevel + 1
  FROM Employee_Table emp
  INNER JOIN cte_a mgr
    ON emp.ManagerID = mgr.EmployeeID
)
SELECT * FROM cte_a
ORDER BY EmployeeLevel
```

Lvl 0 Mark Smith (1007)

Lvl 1 Daniel Jones (1006)

Lvl 1 Michael Lee (1005)

Lvl 2 David Wilson (1004)

Lvl 2 Robert Williams (1002)

Lvl 3 Henry Brown (1003)

Lvl 4 James Clarke (1001)

Window Function — Duplicate Detection

window_function.sql

```sql
WITH cte_bx AS (
  SELECT x, y, z,
    COUNT(*) OVER(
      PARTITION BY x, y
      ORDER BY x
    ) AS cnt
  FROM Sample_1
)
SELECT * FROM cte_bx
WHERE cnt > 1
```

X	Y	Z	cnt
200	400	1	3
200	400	2	3
200	400	3	3
500	600	1	2
500	600	2	2

✦ *cnt > 1 identifies rows where X,Y combo appears multiple times*

Use Case: Identifies duplicate (X, Y) pairs using COUNT() OVER(PARTITION BY) — a window function that counts rows per group without collapsing them like GROUP BY.

Project Summary

3
Core Tables

11
Basic Queries

8
Advanced Queries

2
Special Problems

SQL Server · T-SQL · SSMS

Key Skills

- SELECT / WHERE / DISTINCT / TOP N
- ORDER BY / BETWEEN
- SUM · AVG · COUNT
- GROUP BY + HAVING
- INNER JOIN across 3 tables
- CTE (WITH clause)
- Recursive CTE
- Window Functions (OVER / PARTITION BY)
- Table Creation & INSERT